时间币：一种具备后量子授权的点对点电子现金系统

Williem Doe
jsonrpc@icloud.com

摘要

一种纯粹的点对点电子现金，应当允许在线付款从一方直接送给另一方，而不经过金融机构；并且在出现足以威胁密码学的量子计算机之后，仍然保持有效。数字签名提供了部分解答，但比特币使用的椭圆曲线签名预期无法抵抗Shor算法。本文提出时间币。它是一条名为时间链的新链上的记账单位，代码为TIC。时间链沿用比特币的交易和证明工作量设计，但拥有自己的创世块，不与比特币共享历史。日常授权使用NIST FIPS 204的ML-DSA-65签名。每个地址同时承诺一条FIPS 205的SLH-DSA恢复路径，其安全假设只来自密码学哈希。证明工作量仍是双重SHA-256。Grover算法对哈希只给出平方加速，因此挖矿函数不是第一件必须更换的东西。熟悉的激励、默克尔树和最长链规则予以保留。改变的是锁住硬币的那把锁。

1. 引言

互联网上的商业几乎完全依赖金融机构作为受信任的第三方。中本聪的比特币用一条哈希相连的区块，以及每笔花费必须携带的签名，去掉了支付排序中的这个第三方[1]。这一构造仍然成立。它的授权原语没有同样长的时间尺度。

比特币用secp256k1上的ECDSA，后来又用Schnorr，来锁住硬币。Shor算法可以在足够大的量子计算机上，以多项式时间求解该曲线上的离散对数[2]。一旦公钥已经暴露，无论是已花费输出、重复使用的地址，还是旧式的公钥支付输出，伪造一笔花费在理论上就成为可能。Grover算法作用在SHA-256上是另一件较小的事：它把搜索开方，加长哈希或增加工作量就可以吸收[3]。紧迫的缺陷是签名，不是时间戳。

比特币本身已有若干迁移草案。它们必须携带既有用户、矿工投票，以及公钥已经公开的硬币。时间链没有这些负担。它是一条新账本。时间币是它的单位，代码为TIC。Timechain是中本聪早期笔记里对区块链的叫法，此处作为致敬，不表示对比特币的链、代码或创世块主张权利。比特币持有人不会在时间链上得到余额。没有预挖。创世块的coinbase与比特币一样不可花费，发行从第一笔挖出的补贴开始。

设计目标很窄。保留自2009年公开运行至今的记账模型。把锁换成两种已经标准化的后量子签名，一种快，一种保守，并且一开始就一起承诺，使格假设的破裂不会把硬币困住。规则在第一个区块之前公布，而不是要求一个既有经济体用软分叉把它们接进来。

2. 交易

我们把一枚电子硬币定义为一串数字签名。持有人把硬币转给下一人时，对上一笔交易的哈希和下一位持有人的公钥哈希签名，并把签名附在硬币末尾。收款人可以验证这些签名，从而验证所有权链条。图景与中本聪相同，更换的是签名算法。

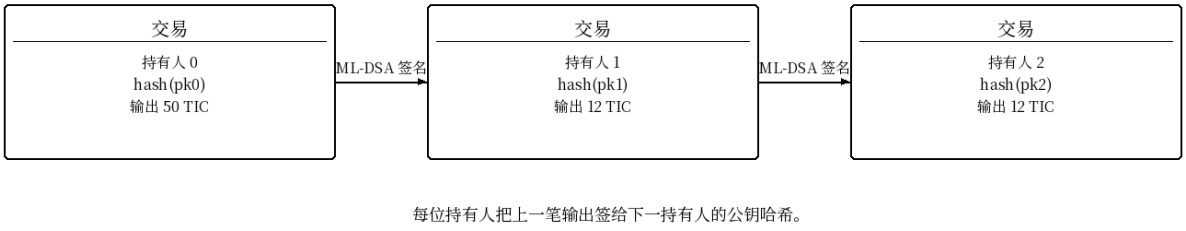


图1. 带后量子签名的时间链交易。

问题在于，收款人无法验证某一位持有人没有把同一枚硬币花费两次。常见办法是设立一个受信任的中央铸币机构检查每一笔交易。我们跟随比特币：交易公开广播，节点对唯一历史达成一致。结构上只增加一件事：被签名的消息承诺签名算法标识，使格签名不能重放到哈希签名程序中，时间链交易也不能重放到比特币上。

输出锁住一段32字节的见证程序。该程序是一棵两叶脚本树的SHA-256哈希。常用叶是ML-DSA-65公钥。恢复叶是SLH-DSA-SHA2-128s公钥。地址不暴露任何一把公钥。花费时揭示所使用的叶子、公钥、签名，以及把叶子重新连回见证程序的默克尔配对哈希。

```
tx_v2 {
  version: 2
  inputs[]  { prevout, sequence }
  outputs[] { value, witness_program }     // 32-byte SHA-256 of script tree
  witness[] { leaf_tag, pubkey, signature, partner_hash }
  locktime
}
sighash = SHA-256d( version | prevouts | outputs | locktime
                    | input_index | algorithm_id | genesis_hash )
```

清单1. 第二版交易，以及做了域分离的签名哈希。代码保持英文。

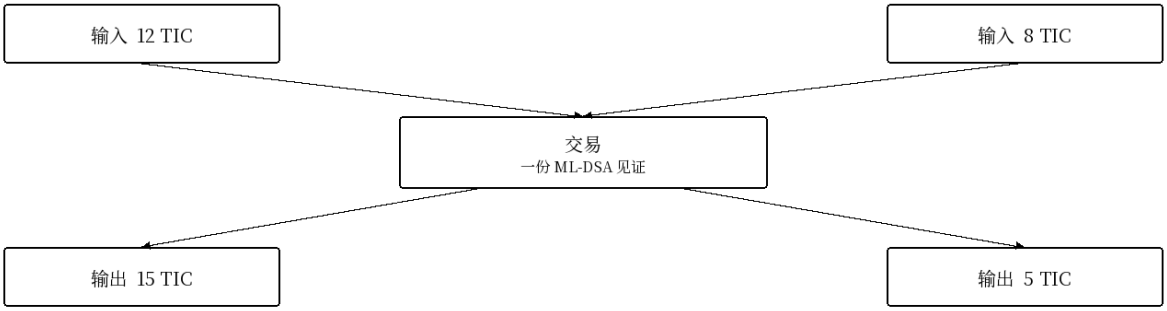


图2. 合并与拆分价值。两笔输入，两笔输出，一份见证。差额是手续费。

3. 时间链

我们提出的办法从一个时间戳服务器开始。时间戳服务器对一批需要盖戳的项目取哈希，并广泛公布该哈希。时间戳证明这些数据在当时必然已经存在，否则无法进入哈希。每个时间戳把前一个时间戳纳入自己的哈希，形成一条链。后加的时间戳加强它之前的那些。

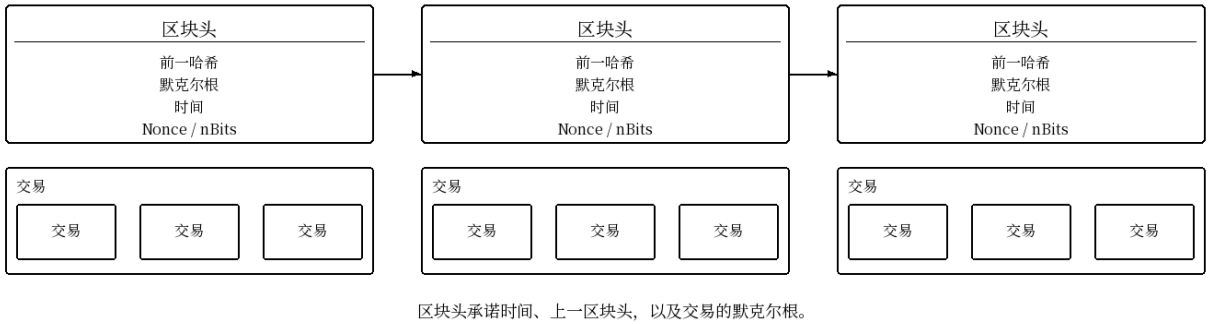


图3. 以区块头哈希相连的时间链区块。

区块头为80字节，布局与比特币相同：版本、上一区块哈希、交易默克尔根、Unix时间、压缩目标和nonce。默克尔根承诺见证交易标识，因此有效签名处在工作量之内。中位时间规则不变。区块时间不得比节点的网络调整时间超前两小时以上。

4. 证明工作量

为了在点对点基础上实现分布式时间戳服务器，我们使用与Adam Back的Hashcash相似的证明工作量[4]，而不是报纸或Usenet帖子。证明工作量是寻找一个值，使它经过双重SHA-256之后，哈希以若干个零比特开始。所需的平均工作量随零比特数指数增长，验证则只需要执行一次哈希。

保留SHA-256是有意的。量子搜索只给出平方加速，不是指数加速[3]。把零比特要求加倍，就可以对Grover对手恢复古典工作量。更换哈希会丢掉每一台现成矿机和每一份经过测试的区块头实现，换来的却不是当前的约束。后量子要改的是签名验证，不是区块头哈希。

目标是60秒一个区块，为比特币间隔的十分之一，因此一笔付款的第一次确认是一分钟而不是十分钟。难度每1008个区块调整一次，大约一天，并使用比特币同样的四倍钳制，使一次失常间隔不能把目标移动超过四倍。最长链是累计工作量最大的链，不是区块数最多的链。一旦处理器、显卡或专用矿机付出工作量满足证明，不重做这些工作就不能改动区块。后面的区块接上之后，改动该区块还必须重做它之后的全部区块。

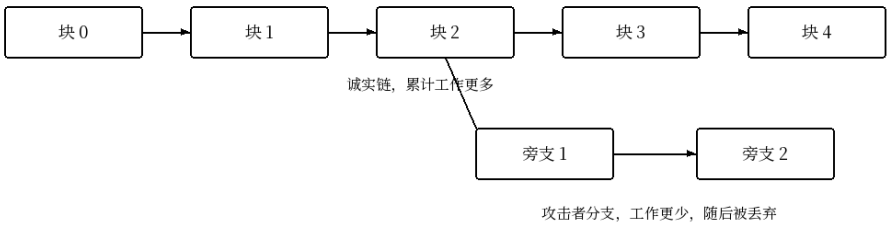


图4. 两条分支。节点先在听到的第一个区块上挖，若另一条分支累计工作更多则改挖那一条。

5. 网络

运行网络的步骤与比特币相同，但使用新的消息起始魔数和新的端口，使时间链节点无法与比特币节点完成握手。

1. 新交易向所有节点广播。

2. 每个节点把新交易收集进一个区块。

3. 每个节点为自己的区块寻找困难的证明工作量。

4. 节点找到证明工作量后，把区块广播给所有节点。

5. 节点只在全部交易有效且未被花费，并且每份见证都能在其声称的算法标签下通过验证时，才接受该区块。

6. 节点用被接受区块的哈希作为下一区块的前一哈希，以此表示接受。

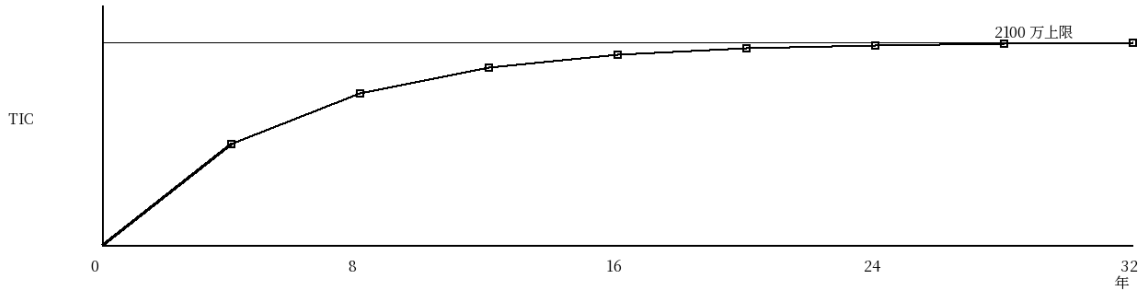
节点始终把累计工作量最大的链视为正确的链，并继续延长它。若两个节点同时广播下一区块的不同版本，有些节点会先收到其中一个。它们先在先收到的那条上工作，但保留另一条分支，以防它变得更长。下一次证明工作量出现、其中一条变得更长时，平局就被打破。

新交易广播不必到达所有节点。只要到达许多节点，它们不久就会进入区块。区块广播也能容忍丢失的消息。节点若没有收到某个区块，会在收到下一区块并发现自己漏掉一块时再去请求。不使用比特币的固定种子。说出比特币魔数的节点会被断开。

6. 激励

按照约定，区块中的第一笔交易是一笔特殊交易，它造出一枚由该区块创造者拥有的新硬币。这为支持网络的节点提供激励，也提供了把硬币初次分配进流通的办法，因为没有中央机构来发行它们。这种稳定增加类似于金矿工消耗资源把黄金加入流通。在这里，消耗的是处理器时间和电力。

补贴从5 TIC开始，每2,100,000个区块减半。以60秒的目标计算，一个纪元是四年。这个几何级数的和是21,000,000 TIC，与比特币上限相同，只是时钟更快。创世输出不在未花费输出集合中。没有创始人分配，也没有金库输出。矿工得到补贴加手续费。



已发行量。补贴每四年减半，级数和为 21,000,000 TIC。

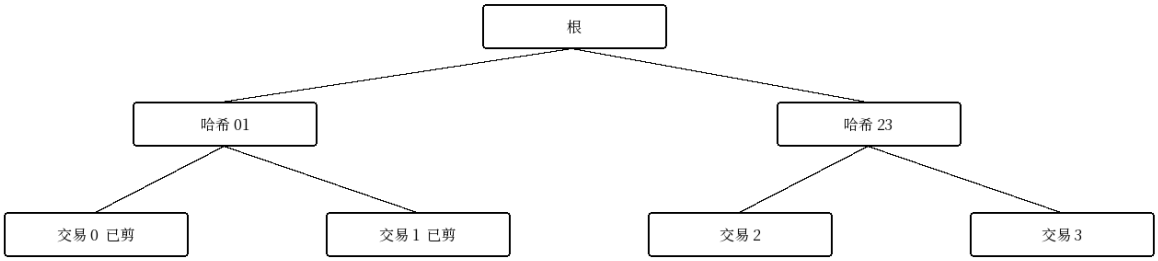
图5. 货币发行日程。每个点是一个四年纪元的结束。

激励也可以由交易手续费提供。若一笔交易的输出价值小于输入价值，差额就是手续费，加入包含该交易的区块的激励。预定数量的硬币进入流通之后，激励可以完全过渡到手续费，从而不再有新增发行。

激励也有助于鼓励节点保持诚实。若一个贪婪的攻击者能够集齐比所有诚实节点更多的证明工作量，他必须在两件事之间选择：用它骗回自己刚付的款，或者用它生成新硬币。他应当会发现，遵守规则、从而比其他所有人合计得到更多新硬币，比破坏系统和他自己财富的有效性更有利。

7. 回收磁盘空间

一枚硬币的最新交易被足够多的区块埋住之后，它之前已经花费的交易可以丢弃，以节省磁盘。为了做到这一点又不破坏区块哈希，交易被哈希进一棵默克尔树，只有根进入区块头。旧区块可以把树的分支剪掉来压缩。内部哈希不必保存。



已花费的叶子可以剪掉。区块头仍然只承诺根。

图6. 剪枝之后的默克尔树。交易0和交易1被剪掉，根不变。

不含交易的区块头大约80字节。以60秒一个区块计算，一年大约4.2 MB，因此即使见证不能保留，区块头仍然可以保留。见证是新的成本。一次ML-DSA-65花费是数千字节，不是64字节。剪掉已花费的见证因此不是优化，而是设计的一部分：节点验证过见证之后，可以在输出已花费且区块已被埋住时丢弃它，同时保留区块头和未花费输出集合。

8. 简化支付验证

不运行完整网络节点也可以验证付款。用户只需要保存最长证明工作量链的区块头副本。他可以向网络节点查询，直到确信自己拥有最长链，并取得把交易连到它所在区块的默克尔分支。他不能自己检查交易，但把它连到链上的一个位置之后，就能看到某个网络节点已经接受它，其后追加的区块进一步确认网络已经接受它。

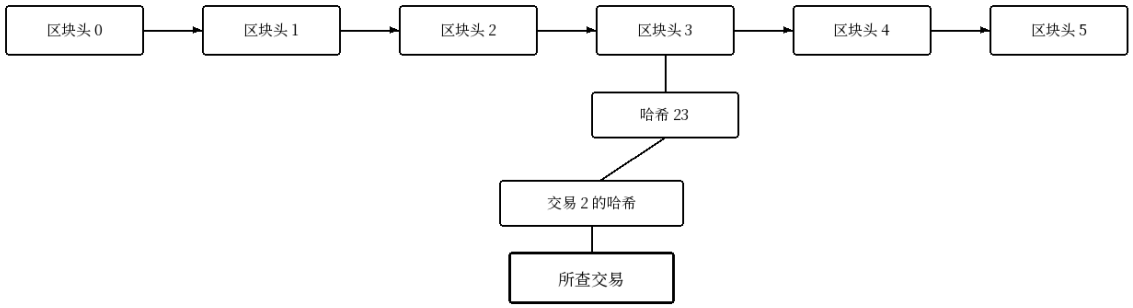


图7. 简化支付验证。区块头加上一条默克尔分支。

因此，只要诚实节点控制网络，这种验证就是可靠的；若网络被攻击者压过，它就更脆弱。频繁收款的商家或轻客户端仍应运行自己的全节点，以获得更独立的安全。轻客户端还必须在第一次确认时取回见证：区块头证明的是包含，不是签名属于后量子算法。若干区块的深度之后，客户端可以丢弃见证，因为逆转它需要重做工作量。

9. 后量子授权

NIST于2024年8月13日公布两项签名标准。FIPS 204标准化ML-DSA，来自CRYSTALS-Dilithium，安全假设是模格[5][6]。FIPS 205标准化SLH-DSA，来自SPHINCS+，安全假设只来自哈希[7][8]。时间链两者都用，分工不同。ML-DSA-65是日常密钥路径：NIST安全类别3，公钥1952字节，签名3309字节。SLH-DSA-SHA2-128s是恢复路径：公钥32字节，签名7856字节，生成较慢，因为它不是常见情形，所以可以接受。



图8. 一个地址同时承诺格路径和基于哈希的恢复路径。

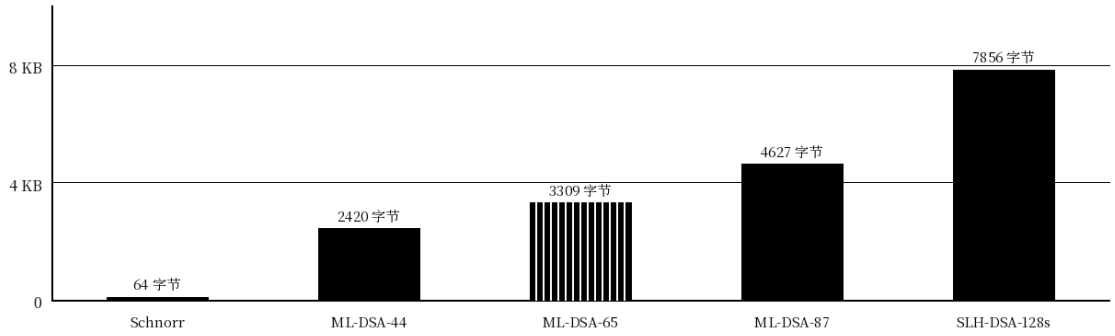


图9. 签名体积。条纹柱是日常的ML-DSA-65花费。Schnorr是被拒绝的成本。

这两条路径不是每次花费都把两种签名接在一起的混合签名。拼接会让普通付款携带庞大的哈希签名。脚本树给出的是密码敏捷性：若模格出现破裂或可信的攻击，钱包就用恢复叶把硬币花到一个新地址，其常用路径本身改为基于哈希，或改为稍后由标志日激活的标准算法。共识从创世块起就理解两个验证器。使用第二条路径不需要软分叉投票。

公钥在花费之前保持隐藏。这关闭了Aggarwal、Brennen、Lee、Santha和Tomamichel指出的那种远程盗窃：比特币上公钥已经在链上的输出可以被事后攻击[9]。它不关闭针对已经进入内存池的花费的短程竞赛。密钥一旦揭示，能在确认窗口内伪造签名的量子对手就可以抢一笔冲突花费。缓解办法是深度。大额收款人等待。钱包不重复使用地址，因为重用会再次公布一把已经打开的密钥。

```
verify(tx, witness):
    leaf = tagged_hash(witness.tag, witness.pubkey)
    root = merkle_pair(leaf, witness.partner)
    require SHA-256(root) == tx.input.witness_program
    if witness.tag == ML-DSA-65:
        require ml_dsa_verify(witness.pubkey, sighash, witness.sig)
    else if witness.tag == SLH-DSA-128s:
        require slh_dsa_verify(witness.pubkey, sighash, witness.sig)
    else reject
```

清单2. 共识验证。每笔花费只接受一个算法标签。代码保持英文。

约束链的是区块权重，不是裸字节。基础区块限制为1,000,000字节。见证字节按四分之一计算，即隔离见证的折扣，使3.3 KB的签名不把一笔付款挤出区块，也使恢复花费仍然可能。折扣之后，一个简单的ML-DSA付款大约相当于一个区块里一百笔普通输入。这是相对比特币有意削减的容量，换来的是一把Shor算法打不开的锁。

参数	比特币	时间链
账本	比特币链，2009 年创世	新创世块，不共享历史
单位 / 代码	bitcoin / BTC	时间币 / TIC
签名	ECDSA，后为 Schnorr	ML-DSA-65，SLH-DSA 恢复
链上公钥	首次花费后常常公开	只出现在花费见证中
证明工作量	SHA-256d，10 分钟	SHA-256d，60 秒
难度调整	2016 个区块	1008 个区块，四倍钳制
补贴	50 BTC，每 21 万块减半	5 TIC，每 210 万块减半
上限	21,000,000 BTC	21,000,000 TIC
预挖	创世输出不可花费	创世输出不可花费
地址	bc1…	tic1… bech32m

表1. 从比特币复制的部分，以及没有复制的部分。

10. 隐私

传统银行模型通过把信息限制在当事人和受信任第三方之内来获得一定隐私。公开宣布所有交易使这种方法不再可用，但隐私仍可以在另一处切断信息流来维持：让公钥保持匿名。公众可以看见某人把一笔金额送给另一个人，但没有把交易连到任何人的信息。这是比特币画出的同一道防火墙。

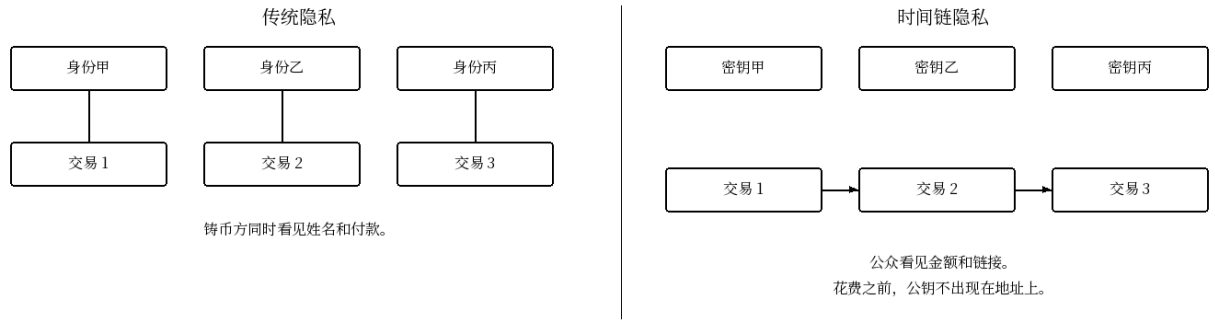


图10. 隐私从受信任的一方，移到密钥和交易之间的空隙。

后量子密钥使卫生要求更严，而不是更松。一把ML-DSA公钥一旦打开，就是1952字节的识别材料。钱包必须为每一笔输出使用新的常用密钥和新的恢复密钥。密钥派生不能使用BIP-32的椭圆曲线调整。链码把SHA-256送入ML-DSA和SLH-DSA的密钥生成器，派生本身留在链下。公布重复地址的收款人，等于公布一把对手可以存档、等到量子计算机出现那天使用的密钥。作为额外防火墙，每笔交易应使用新的密钥对，以免它们被连到同一个持有人。多输入交易仍不可避免地暴露其输入属于同一持有人。风险在于，若一把密钥的持有人被揭示，链接可能揭示属于同一持有人的其他交易。

11. 计算

我们考虑攻击者试图生成一条比诚实链更快的替代链。即使他做到了，系统也不会因此允许任意改动，例如凭空创造价值，或拿走从未属于攻击者的钱。节点不会把无效交易当作付款接受，诚实节点也永不接受包含它们的区块。攻击者只能试图改掉他自己的某一笔交易，以拿回他最近花出的钱。

诚实链和攻击者链之间的竞赛可以描述为二项随机游走。成功事件是诚实链延长一个区块，领先增加1。失败事件是攻击者链延长一个区块，差距减少1。攻击者从给定落后中追上的概率，类似于赌徒破产问题。假设一个信用无限的赌徒从亏空开始，可以进行潜在无限次试验来试图回到盈亏平衡。他最终回到盈亏平衡的概率，也就是攻击者最终追上诚实链的概率，是：

$$q_z = 1 \text{ , if } p \le q$$
$$q_z = (q / p)^z \text{ , if } p > q$$

p是诚实节点找到下一区块的概率，q是攻击者找到下一区块的概率，q_z是攻击者从落后z个区块最终追上的概率。

我们假设p大于q。攻击者必须追上的区块越多，概率就下降得越快。在胜算不利于他时，若他不能在早期幸运地向前冲一步，随着进一步落后，他的机会就变得极小。收款人等到交易进入一个区块，并且其后又接上z个区块。他不知道攻击者的确切进度，但假设诚实区块花费了平均预期时间，攻击者的可能进度就是一个泊松分布，期望值为：

$$\lambda = z \, (q / p)$$

为了得到攻击者现在仍能追上的概率，把他可能做出的每一种进度的泊松密度，乘以他从该点追上的概率：

$$1 - \Sigma_k{=}0..z \, [\, \lambda^k \, e^{(-\lambda)} / k! \,] \times [\, 1 - (q/p)^{(z-k)} \,]$$

在60秒的时钟上，z是区块数，不是分钟数。六次确认是六分钟，不是一小时。相对q的衰减不变，因为它取决于相对哈希率，不取决于墙上时间。想要比特币那一小时等待的商家，仍然等待60个区块。

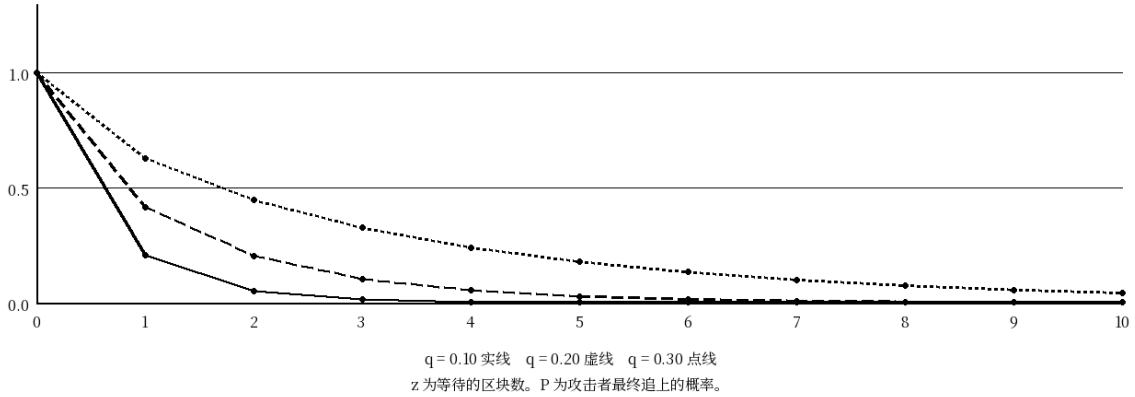


图11. 哈希份额为q的攻击者最终追上已有z次确认的交易的概率。实线、虚线、点线分别是q = 0.10、0.20、0.30。

q \ z	**1**	**2**	**3**	**4**	**5**	**6**	**10**
0.10	0.2046	0.05098	0.01317	0.00346	0.00091	0.00024	< 0.00001
0.15	0.3212	0.1220	0.04818	0.01937	0.00785	0.00319	0.00009
0.20	0.4512	0.2176	0.1076	0.05354	0.02670	0.01331	0.00081
0.30	0.7326	0.5113	0.3598	0.2536	0.1787	0.1259	0.03061
0.40	0.9531	0.8283	0.7063	0.5950	0.4990	0.4161	0.1908

表2. 追上概率。q = 0.1时，六个区块之后P小于0.001。

这些数字计算的是古典攻击者。一个没有攻破SHA-256的量子攻击者，仍然必须在工作量上赢得这场竞赛。一个已经攻破ML-DSA、并且在内存池里看见密钥的量子攻击者，不在这张表里：他是在伪造，不是在比哈希。恢复叶存在，就是为了这种情况。大额收款即使第一份签名已经验证，也要等待深度才视为最终。

12. 结论

我们提出了一种电子交易系统。它的排序不依赖信任，它的授权不依赖椭圆曲线。框架是熟悉的：由签名做成的硬币，由工作量证明的点对点时间戳服务器，以及以工作量最长的延伸作为历史的链。增加的是一把为量子对手写成的标准所选定的锁，以及一把不与第一把锁共享假设的第二把锁。

时间链不是比特币账本的分叉。它是一条新链，有新的创世块、新的端口和新的地址族。这样后量子规则可以从第一笔输出起即为真，而不必在已经暴露的硬币上分期引入。项目创始人是Williem Doe。链不会因为他是创始人而付给他预挖。节点可以离开并重新加入网络，把证明工作量链当作它们离开期间发生之事的证明，并拒绝任何见证不能通过验证的区块。

本文说明的是一项设计。它不报告已经启动的主网、已经交易的市场或已经完成的审计。参数数字遵循已公布的FIPS 204和FIPS 205。实现必须调用经过审查的库来做两种验证，而不是私下重新实现格或哈希树。

参考文献

[1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. https://bitcoin.org/bitcoin.pdf

[2] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” Proceedings of the 35th Annual Symposium on Foundations of Computer Science, pp. 124–134, 1994.

[3] L. K. Grover, “A fast quantum mechanical algorithm for database search,” Proceedings of the 28th Annual ACM Symposium on Theory of Computing, pp. 212–219, 1996.

[4] A. Back, “Hashcash — a denial of service counter-measure,” technical report, August 2002.

[5] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation,” NIST PQC round-3 submission, 2021.

[6] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” FIPS 204, August 2024.

[7] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, “The SPHINCS+ Signature Framework,” Cryptology ePrint Archive, Report 2019/1086, 2019.

[8] National Institute of Standards and Technology, “Stateless Hash-Based Digital Signature Standard,” FIPS 205, August 2024.

[9] D. Aggarwal, G. K. Brennen, T. Lee, M. Santha, and M. Tomamichel, “Quantum attacks on Bitcoin, and how to protect against them,” Ledger, vol. 3, 2018.